Doc. no.	P0914R1
Revises	P0914R0
Date:	2018-03-15
Reference:	ISO/IEC TS 22277, C++ Extensions for Coroutines
Audience:	EWG
Reply to:	Gor Nishanov <gorn@microsoft.com>

Add parameter preview to coroutine promise constructor

Issue

Users of C++ coroutines have long been requesting an ability to access coroutine parameters in the constructor of the coroutine promise. Current workaround is to override `operator new` of the coroutine promise that does allow observing of coroutine parameters, then extract the value of the desired parameter and store it in a `thread_local` variable and later extract the value from the `thread_local` in the coroutine promise default constructor and store the value in the coroutine promise.

The workaround used is not reliable as compiler is allowed to elide heap allocation for the coroutine state and elide invocation of `operator new`. Coroutines need direct and reliable way of expressing the desired behavior.

Suggestion is to give a coroutine promise an ability to look at the coroutine parameters.

Before:

```
struct my_promise_type {
  static thread_local cancellation_token saved_tok;
  cancellation_token tok;

  template <typename... Whatever>
  void* operator new(size_t sz, cancellation_token tok, Whatever const&...) {
    // BROKEN: May not get called if heap allocation is elided.
    saved_tok = tok;
    return ::operator new(sz);
  }

  my_promise_type() : tok(saved_tok) {}
  ...
};
```

After:

```
struct my_promise_type {
  cancellation_token tok;

  template <typename... Whatever>
  my_promise_type(cancellation_token tok, Whatever const&) : tok(tok) {}
  ...
};
```

Wording:

[Proposed wording is relative to N4723].

Modify paragraph 8.4.4/3 as follows:

> For a coroutine *f* that is a non-static member function, let *P1* denote the type of the implicit object parameter (13.3.1) and *P2* ... *Pn* be the types of the function parameters; otherwise let *P1* ... *Pn* be the types of the function parameters. Let *p1* ... *pn* be lvalues denoting those objects. Let *R* be the return type and *F* be the function-body of *f*, *T* be the type `std::experimental::coroutine_traits<`*R,P1,...,Pn*`>`, and *P* be the class type denoted by *T*`::promise_type`. Then, the coroutine behaves as if its body were:
>
> ```
> {
>   P p promise-constructor-arguments;
>   co_await p.initial_suspend(); // initial suspend point
>   try {
>     F
>   } catch(...) { p.unhandled_exception(); }
> final_suspend:
>   co_await p.final_suspend(); // final suspend point
> }
> ```
>
> where an object denoted as *p* is the promise object of the coroutine, ~~and~~ its type *P* is the *promise type* of the coroutine, and *promise-constructor-arguments* is determined as follows: overload resolution is performed on a promise constructor call created by assembling an argument list with lvalues p_1 ... p_n. If a viable constructor is found (16.3.2), then *promise-constructor-arguments* is (p_1,...,p_n), otherwise *promise-constructor-arguments* is empty.

Modify paragraph 7 of 8.4.4/7 as follows

> An implementation may need to allocate additional storage for a coroutine. This storage is known as the coroutine state and is obtained by calling a non-array allocation function (3.7.4.1). The allocation function's name is looked up in the scope of *P*. If this lookup fails, the allocation function's name is looked up in the global scope. If the lookup finds an allocation function in the scope of *P*, overload resolution is performed on a function call created by assembling an argument list. The first argument is the amount of space requested, and has type `std::size_t`. The lvalues *p1*...*pn* are the succeeding arguments. If no viable~~matching~~ function is found (16.3.2), overload resolution is performed again on a function call created by passing just the amount of space required as an argument of type `std::size_t`.

Add underlined text to 8.4.4/11:

> When a coroutine is invoked, a copy is created for each coroutine parameter. Each such copy is an object with automatic storage duration that is direct-initialized from an lvalue referring to the corresponding parameter if the parameter is an lvalue reference, and from an xvalue referring to it otherwise. A reference to a parameter in the function-body of the coroutine and in the call to the coroutine promise constructor is replaced by a reference to its copy.

Implementation experience

Implemented in clang trunk.